ABDALLAH KASSAN

Ingénieur logiciel Full Stack / Backend / Cloud / Edge AI

Caen, France | Mobilité en France | +33 7 88 09 35 91 | abdallah.kassan.job@outlook.fr
linkedin.com/in/kassan-abdallah | github.com/kassanabdallah0 | gitlab.com/abdallah.kassan

PROFIL

Ingénieur diplômé de l'Université de Technologie de Troyes (UTT), avec **plus de deux ans en CDI** chez Fastpoint et deux stages de développement logiciel. Conception et livraison de produits de bout en bout : **React/TypeScript**, **Node.js/TypeScript**, **Python/Flask**, API REST, architecture hexagonale, tests automatisés, AWS et GitLab CI/CD. Expérience opérationnelle de plateformes cloud, de systèmes IoT et de vision par ordinateur sur **NVIDIA Jetson**, du besoin utilisateur jusqu'à la maintenance en production.

RÉSULTATS MARQUANTS

- **Autonomie client :** configuration des zones SecuriSPOT passée de **30–60 min à moins de 2 min** grâce à une interface en libre-service (estimations terrain).
- **Vision industrielle :** pipeline d'analyse optimisé d'environ **5 à 30 FPS** (×6) ; **plus de 80 % de fausses alertes en moins**, selon observations terrain.
- **Cloud / IoT :** migration de **70 bus** vers AWS, supprimant **70 PC embarqués** ; réduction estimée des déplacements de maintenance d'environ **92–94 %**.
- **Livraison Edge :** déploiement de code sur Jetson déjà provisionné ramené d'environ **3 h à moins de 5 min** ; retour automatique à une version fonctionnelle en **moins de 2 min** dans les cas récupérables.

EXPÉRIENCE PROFESSIONNELLE

FASTPOINT – Ingénieur logiciel Full Stack **Sept. 2024 – aujourd'hui**
CDI – Développement produit, backend, cloud, Edge AI et DevOps Caen

SecuriSPOT Manager – application web de supervision et de configuration industrielle

- **Conçu et développé de bout en bout** l'application **React, TypeScript, MUI, i18n** et son backend **Python/Flask**, utilisée sur environ **5 à 6 installations** : UX/UI, composants, API REST, configuration et supervision.
- Réalisé la **visualisation des flux caméra et des résultats de détection IA** (classes, boîtes englobantes, zones), les tableaux de bord d'alarmes et de règles de sécurité enfreintes, à partir des journaux de détection sur Jetson exposés par Flask.
- Développé une **configuration graphique des zones d'intérêt et des paramètres de détection**, permettant aux clients de modifier eux-mêmes les zones en **moins de 2 min**, contre **30–60 min** avec intervention technique auparavant (estimations terrain).
- Conçu l'**historique des alarmes** : captures enregistrées localement sur Jetson, métadonnées associées (bounding boxes, zones), classement, miniatures et **lazy loading au défilement**.
- Ajouté des **notifications dans l'interface ouverte** pour erreurs, avertissements et pertes de connexion, via **polling HTTP** des API Flask ; réalisé un **guide utilisateur animé** et les parcours de paramétrage.

COLAS – Scan EPI interactif sur télévision industrielle

- Identifié les limites d'une première version exécutée dans le navigateur de la TV : affichage d'environ **2–3 FPS**, pénalisant les interactions pendant les différentes phases du scan EPI.
- **Proposé et développé une nouvelle architecture** exécutant l'application **React/TypeScript sous Chromium/Weston sur Jetson**, avec **affichage HDMI sur la TV** et diffusion vidéo locale par **GStreamer/WebRTC**.
- Déplacé les **décisions et états métier du scan** dans le backend ; conçu les interfaces de stabilisation, acquisition, validation et restitution du résultat. **Affichage final réglé à 10 FPS** (contre 2–3), avec 15–20 FPS observés lors des essais.
- Rendu l'application **accessible au diagnostic et à la mise à jour à distance via Jetson**, contrairement aux limitations de maintenance rencontrées sur la TV ; intégration au processus de release automatisé. *La détection par modèle IA était développée par un collègue.*

FASTPOINT – VISION INDUSTRIELLE ET SYSTÈMES EDGE

SecuriSPOT – traitement vidéo, règles de sécurité et alertes

- Contribué au développement et à l'optimisation du **backend Python/Flask multiprocessus** sur **NVIDIA Jetson Orin**, avec caméra **Intel RealSense**, OpenCV, modèles **YOLO/Ultralytics** et inférence **TensorRT**.
- **Porté le débit de traitement d'environ 5 à 30 images/s (×6)** sur un pipeline de vision industrielle : substitution de segmentation par détection d'objets lorsque les masques étaient inutiles, optimisation des exports YOLO/TensorRT et réduction des échanges interprocessus.
- Réduit les transferts coûteux d'images et de masques entre processus en privilégiant les **identifiants de frames, classes et bounding boxes** ; récupération/compression des images uniquement lorsque nécessaire pour une alarme ou l'affichage.
- Développé et ajusté des **règles de détection temporelle par suivi d'objet**, avec fenêtre FIFO de l'ordre de 30 frames et seuils de confirmations paramétrables (3, 6 ou 10 observations), pour limiter les déclenchements intempestifs.
- Obtenu une **réduction estimée de plus de 80 % des fausses alertes** dans les scénarios observés ; délai estimé entre événement et activation d'un dispositif d'alerte (gyrophare / panneau lumineux) ramené d'environ **1–1,5 s à 100–250 ms**. *Estimations terrain, sans benchmark instrumenté.*
- Intervenu sur des cas d'usage de **sécurité industrielle** : port d'EPI, présence en zone, comportements à risque, proximité véhicules/personnes, collisions potentielles et comptage selon les projets clients.
- Participé aux travaux de **préparation de jeux de données et d'intégration de modèles** (Roboflow, PyTorch, Ultralytics, export TensorRT) ; adaptation de modèles et de configurations aux contraintes des équipements et sites.

Déploiements Jetson – GitLab CI/CD, Docker et fiabilité des releases

- **Conçu une chaîne de livraison versionnée** : création d'un **tag Git** déclenchant automatiquement GitLab CI/CD, exécution des tests **Jest et pytest**, préparation de la release et publication d'images dans **GitLab Container Registry**.
- Mis en place un **déploiement contrôlé sur chaque Jetson** : commande locale paramétrée sélectionnant la version, puis exécution automatisée des vérifications et de la mise à jour. *Le déploiement final n'est pas déclenché automatiquement à la fin de la CI.*
- Automatisé les vérifications matérielles et logicielles : **caméra RealSense/USB**, images Docker, dépendances, exports de modèles, journaux, **health checks Docker**, smoke tests et vérification du bon fonctionnement applicatif.
- Intégré la **gestion des services Linux** : synchronisation horaire (NTP), rotation des logs, suppression automatisée des anciennes captures/métadonnées d'alarmes avec une **rétention de 7 jours** (environ 50–200 alarmes/semaine selon les sites).
- Développé un **rollback automatique** en cas de release défectueuse récupérable, avec retour à la version précédente et vérifications : **moins de 2 min** pour restaurer le service dans ces situations, contre des interventions manuelles pouvant durer plusieurs heures.
- **Réduit le déploiement d'une mise à jour du code de 3 h à moins de 5 min** sur un Jetson déjà provisionné ; pour une première installation, temps ramené d'environ **1–2 jours à au plus une demi-journée**, hors variations liées aux téléchargements d'images et exports des modèles (estimations terrain).
- Géré les releases de **plusieurs produits personnalisés** pour COLAS, IDEA, SNCF et autres installations, avec **environnement Docker commun**, branches/tags spécifiques et exports de modèles adaptés à chaque projet ; il ne s'agit pas d'une base de code unique entièrement paramétrable.

Qualité logicielle et diagnostic terrain

- Écrit des **tests unitaires et fonctionnels Jest/pytest** pour les API Flask et le frontend React, exécutés en **CI** avec blocage des releases en cas d'échec.
- Investigations en production sur **Linux/Jetson, Docker, Flask, React, TensorRT, caméra USB et Chromium/Weston** : analyse des logs, problèmes de modèles/compatibilité, régressions d'affichage et diagnostic à distance.

FASTPOINT – MEDIASPOT : PLATEFORME CLOUD ET BUS CONNECTÉS

Migration de 70 bus vers AWS – architecture et exploitation

- **Migré une solution de portail Wi-Fi embarqué de 70 bus Cap Cotentin** depuis une architecture avec PC local dans chaque véhicule vers un environnement **centralisé sur AWS**, supprimant **70 PC embarqués**.
- Déployé et configuré des **routeurs Teltonika RUTX11** connectés à un **serveur OpenVPN sur AWS EC2**, avec authentification par certificats, accès distant et exploitation du portail web pour les passagers.
- Pris en charge la **configuration AWS et réseau** : EC2, VPC, Security Groups, règles de pare-feu, routage, certificats VPN et connectivité entre équipements et services ; ressources configurées dans **AWS Console**.
- **Réduit les déplacements de maintenance** d'environ 2–3 interventions par mois à une intervention tous les six mois (**92–94 %**, estimation terrain) grâce à la disparition des PC embarqués et aux outils de diagnostic à distance ; une période ultérieure d'environ **12 mois sans visite** a été rapportée.
- Développé une **supervision des routeurs** fondée sur des événements SMS au démarrage, un routeur central et des commandes SMS de diagnostic (état Wi-Fi, connexions, informations réseau), complétées par le suivi OpenVPN et l'outil Teltonika RMS selon les besoins.

Backend TypeScript/Node.js – architecture hexagonale et sécurité

- **Conçu et structuré personnellement le backend TypeScript/Node.js** selon une **architecture hexagonale (ports et adaptateurs)**, avec séparation des couches domaine, application et infrastructure ; contrats d'API typés et documentation technique maintenue séparément.
- Développé des **API REST et fonctions AWS Lambda** pour l'accès aux contenus, la supervision des routeurs et le reporting, avec intégration d'**AWS API Gateway**, CloudWatch et des autres services de la plateforme.
- Mis en place l'**authentification JWT** des utilisateurs applicatifs, des **autorisations AWS IAM sur des endpoints API Gateway** et des rôles/politiques IAM contrôlant les accès entre services AWS.
- **Automatisé le déploiement des Lambda** par des scripts **Bash exécutant AWS CLI** ; infrastructure AWS configurée principalement via Console (*pas de revendication Terraform/CloudFormation/CDK*).

Portail React, consultation des journaux et reporting mensuel

- **Développé le portail React** accessible depuis le réseau Wi-Fi des bus et un **lecteur de journaux numériques conçu de zéro**, avec chargement progressif des pages PDF au défilement ; publications de **25 à 40 pages**.
- Intégré les contenus hébergés sur **Amazon S3** à l'aide des services backend/Lambda. *Le mécanisme initial d'alimentation des journaux dans S3 existait déjà et n'a pas été développé par le candidat.*
- **Refondu entièrement le reporting client existant** en **TypeScript/Node.js et AWS Lambda** : interrogation des logs **CloudWatch**, agrégation des connexions Wi-Fi, volumes consommés et consultations de journaux, génération d'un **rapport PDF mensuel**.
- Planifié l'exécution avec **Amazon EventBridge** et automatisé l'envoi des rapports via **Amazon SES**, en supprimant la dépendance du reporting aux PC embarqués. *Le reporting précédent était déjà automatique, mais fragile et lié à l'ancienne architecture.*

COMPÉTENCES MISES EN OEUVRE DANS MEDIASPOT

Full Stack : React, TypeScript, Node.js, API REST, JWT, architecture hexagonale, lecteur PDF et chargement différé.

Cloud / réseau : AWS EC2, Lambda, API Gateway, S3, CloudWatch, EventBridge, SES, IAM, VPC, OpenVPN, TLS, Teltonika RUTX11/RMS.

Livraison : Bash, AWS CLI, diagnostics distants, journaux applicatifs et VPN.

FASTPOINT – EXPLOITATION ELK ET SÉCURITÉ

- **Supervisé et maintenu un cluster Elasticsearch/Logstash/Kibana (ELK) de 3 nœuds**, ayant historiquement atteint environ **306 millions de documents et 644 Go** de données.
- Réalisé les **opérations de nettoyage et de maintenance du stockage** ; configuré les politiques **Elasticsearch ILM** pour supprimer automatiquement les anciens index après **180 jours**.
- Créé et configuré les **certificats TLS** pour sécuriser l'accès HTTPS à Elasticsearch/Kibana et les **communications inter-nœuds** du cluster.
- Les volumes historiques étaient en partie dus à des logs applicatifs trop nombreux ; *la correction de ces émissions côté SecuriSPOT a été effectuée par un collègue.*

AUTRES EXPÉRIENCES PROFESSIONNELLES

KEOS Telecom – Développeur web Full Stack **Juil. – déc. 2023**
Stage – Application métier télécom Paris

- Développé et maintenu une **application métier interne PHP/MySQL** : évolutions frontend et backend, requêtes SQL, corrections d'anomalies, tests et documentation technique.

HMRexpert – Développeur logiciel / traitement d'image **Fév. – juil. 2022**
Stage – Développement Linux et vision Bourgogne-Franche-Comté

- Développé une **application Qt sous Linux** et des modules de **traitement d'image OpenCV** ; intégration, tests et documentation technique.

COMPÉTENCES TECHNIQUES – INVENTAIRE DE RÉFÉRENCE

Frontend et UX : React, TypeScript, JavaScript, HTML/CSS, MUI, SCSS, Webpack, i18n, interfaces interactives, tableaux de bord, lazy loading, WebRTC, Chromium/Weston.

Backend et architecture : Node.js, Express, Python, Flask, API REST, contrats TypeScript, architecture hexagonale (ports/adaptateurs), PHP, traitement asynchrone, intégration IoT.

Données et observabilité : SQL, PostgreSQL, MySQL, DynamoDB, Elasticsearch, Logstash, Kibana, CloudWatch, analyse de logs, gestion des index et rétention ILM.

AWS et réseau : EC2, Lambda, API Gateway, S3, CloudFront, Route 53, CloudWatch, EventBridge, SES, IAM, VPC, SNS, SQS, OpenVPN, TLS/certificats, pare-feu et routage.

DevOps, Linux et qualité : Docker, Git, GitLab CI/CD, GitLab Container Registry, Bash, AWS CLI, Jest, pytest, smoke tests, health checks, rollback, logrotate, NTP, supervision.

IA, vidéo et embarqué : NVIDIA Jetson Orin/Xavier, Intel RealSense, OpenCV, Roboflow, PyTorch, Ultralytics/YOLO, TensorRT, GStreamer, WebRTC, suivi d'objets, optimisation de pipelines vidéo.

Compétences complémentaires : Kafka, Nginx ; notions de C/C++ et Qt acquises en formation/projets, sans revendiquer d'expérience professionnelle de développement firmware C/C++.

FORMATION ET CERTIFICATIONS

Diplôme d'ingénieur – Université de Technologie de Troyes (UTT) **2021 – 2024**
Systèmes de Réseaux Convergents

TensorFlow – certifications Coursera : certificat professionnel et spécialisation.

LANGUES ET APPROCHE PROFESSIONNELLE

Français : professionnel | **Anglais :** professionnel | **Arabe :** langue maternelle.

Méthode de travail : autonomie, résolution de problèmes, approche produit, communication technique, collaboration et diagnostic en production.

Document maître de référence, volontairement exhaustif et non destiné à être envoyé tel quel. Les durées, FPS et gains identifiés comme estimations terrain proviennent des retours du candidat ; ils ne sont pas présentés comme des benchmarks indépendants. Pour chaque candidature, sélectionner les réalisations pertinentes et produire un CV ciblé.